

## 06 · 4:1 멀티플렉서

VS Code 사전 시뮬레이션 → Vivado 2026.1 → 보드 실험

선택  $s=00,01,10,11$ 은 각각  $i[3], i[2], i[1], i[0]$ 을  $z$ 로 보낸다. 원본의 비트 순서를 따른다.

1. 템플릿을 clone하고 이 회로의 workspace를 연다.
2. 예상값·코드·테스트벤치를 읽고 시뮬레이션한다.
3. Vivado 결과와 실제 보드 동작을 비교한다.
4. 자신의 사진·영상·레포트를 GitHub에 연결한다.

작성일 2026. 09. 10. · 이해리

# 실습 목차

1부 · VS Code 사전 실습

새 창·workspace·확장·코드 열기

예상값·작업 실행·로그·파형·실험 전 레포트

2부 · Vivado 실습

프로젝트·소스·top·시뮬레이션·핀·비트스트림

3부 · 결과 정리

보드·사진·영상·GitHub·실험 후 레포트

전체 목차 PDF 열기

# 준비할 프로그램

Git·Python 3·VS Code·slang-server·VaporView를 준비한다.

Vivado GUI를 열기 전에도 XSim 도구가 필요하므로 Vivado는 미리 설치한다.

```
git -version / python -version / code -version
```

```
vivado -version / xvlog -version / xelab -version / xsim -version
```

명령이 없으면 설치와 PATH 또는 VIVADO\_BIN을 확인하고 VS Code를 다시 연다.

Vivado 설치 PDF VS Code 환경설정 PDF

# 별도 템플릿 clone

PowerShell에서 실습 폴더를 둘 위치로 이동한 뒤 실행한다.

```
git clone https://github.com/Glaysia/fpga-lab-template.git
```

```
cd fpga-lab-template
```

자신의 저장소를 만들려면 GitHub의 Use this template → Create a new repository를 사용하고 자신의 주소를 clone한다.

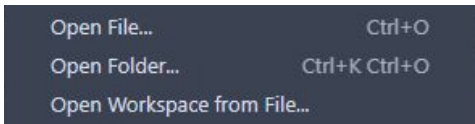
학생용 템플릿 열기

# File → New Window

VS Code 상단 File → New Window를 누른다. Ctrl+Shift+N도 같은 동작이다.



새 창에서 File → Open Workspace from File...을 선택한다.



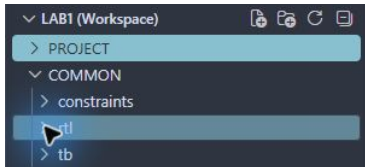
Open File...로 코드 하나만 여는 것과 구분한다.

# 이 회로의 workspace 선택

clone한 폴더에서 다음 경로로 이동한다.

vivado\_2026\_1/06\_mux4to1

vivado\_2026\_1\_06\_mux4to1.code-workspace



PROJECT는 이 회로의 실행 설정과 결과, COMMON은 공유 RTL·TB·XDC다.

위 화면은 공통 폴더 구조 예이며 선택할 파일명은 이 슬라이드의 경로를 따른다.

# 확장 설치 · 활성화

Extensions에서 slang-server와 VaporView를 검색한다.



**slang-server: Verilog/SystemVerilog LSP**  
Hudson River Trading | 3,898 | ★★★★★ (5)

Verilog and SystemVerilog support via the slang language server.

Update to v0.3.0 Enable Disable Uninstall ✓ Auto Update

This extension is enabled for this workspace by the user.  
 ⓘ This extension is not updated yet because new versions are auto updated 12 hours after they are published.  
 It will be auto updated in 11 hrs.

없으면 Install, 꺼져 있으면 Enable 또는 Enable (Workspace).

Verilog 구문 강조가 보여도 시뮬레이션이 통과했다는 뜻은 아니다.

# Explorer에서 파일 열기

1. PROJECT와 COMMON 왼쪽 화살표를 눌러 펼친다.
  2. RTL: mux\_4x1.v. 파일을 두 번 누르면 탭으로 열린다.
  3. 검증 TB: tb\_mux\_4x1.sv.
  4. 핀 제약: mux\_4x1.xdc.
  5. 수정 후 File → Save All. 저장한 뒤에 시뮬레이션한다.
- 이 프로젝트 소스·workspace 열기

# 입출력과 동작 규칙

입력  $i[3:0]$ ,  $s[1:0]$  / 출력  $z$

선택  $s=00,01,10,11$ 은 각각  $i[3], i[2], i[1], i[0]$ 을  $z$ 로 보낸다. 원본의 비트 순서를 따른다.

$KEY1-4=i[3:0]$ ,  $DIP1-2=s[1:0]$ .  $LED1=z$ .

# 실행 전에 예상값 작성

| 입력 또는 조건     | 예상 결과 |
|--------------|-------|
| i=1000, s=00 | z=1   |
| i=1000, s=01 | z=0   |
| i=0001, s=11 | z=1   |

i=1000인 320–360ns에서 s를 바꿨을 때 선택된 입력을 추적한다.  
예상값을 계산한 근거를 자신의 말로 설명한다.

# RTL 구조를 설명한다

실제 배포 RTL을 VS Code에서 연 화면이다.

설계 top: mux\_4x1

```
1 module mux_4x1(input wire [3:0] i, input wire [1:0] s, output wire z);  
2     // Preserve the original lab ordering: 00 -> i[3].  
3     assign z = i[3-s];  
4 endmodule
```

선택 s=00,01,10,11은 각각 i[3],i[2],i[1],i[0]을 z로 보낸다. 원본의 비트 순서를 따른다.

포트 선언·비트 폭·출력 연결을 짚어 설명한다. 저장소 소스를 복사해 사용해도 된다.

배포 소스 확인

# 테스트벤치와 통과 기준

시뮬레이션 top: tb\_mux\_4x1

64개 입력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 640ns다.

기대값과 실제 출력이 다르면 \$fatal로 중단한다. 검사 횟수와 watchdog도 확인한다.

통과 문구: LAB1\_PASS mux\_4x1 cases=64

원본 레거시 testbench.v와 별도의 자기검사 TB는 파일과 역할을 구분한다.

## TB · 입력과 기대값

tb\_mux\_4x1.sv · 1-12행 · 실제 VS Code 화면

```
1 | timescale 1ns/1ps
2 | module tb_mux_4x1;
3 |     reg [3:0] i; reg [1:0] s; wire z; mux_4x1 dut(i,s,z);
4 |     integer n,checked=0;
5 |     reg [0:0] expected;
6 |     initial begin
7 |         $dumpfile("wave.vcd"); $dumpvars(0,tb_mux_4x1);
8 |
9 |         for(n=0;n<64;n=n+1) begin
10 |             {i,s}=n;
11 |             expected=((n/4) >> (3-(n%4))) & 1;
12 |             #10;
```

입력 i의 16개 값과 선택선 s의 4개 값을 조합한다. s=0이 i[3]을 고르는 순서에 주의한다.

DUT는 검사할 회로다. dumpfile·dumpvars는 파형 파일에 신호를 남긴다.

## TB · 비교와 종료

tb\_mux\_4x1.sv · 13-22행 · 실제 VS Code 화면

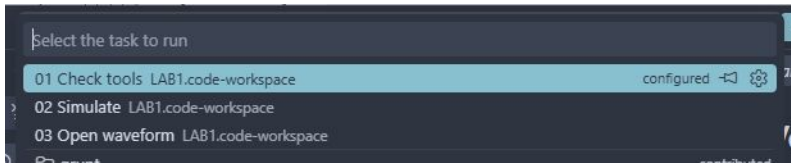
```
13     if (z !== expected)
14         $fatal(1,"FAIL mux_4x1 vector=%0d expected=%h actual=%h",n,expected,z);
15     checked=checked+1;
16 end
17 if(checked!=64) $fatal(1,"Incomplete test");
18 $display("LAB1_PASS mux_4x1 cases=%0d",checked);
19 $finish;
20 end
21 initial begin #100000; $fatal(1,"Watchdog"); end
22 endmodule
```

!==는 X·Z를 포함한 불일치도 잡는다. 다르면 \$fatal로 중단하고 어떤 입력에서 틀렸는지 출력한다.

checked가 전체 검사 수와 같은지 확인한 뒤 PASS와 \$finish. watchdog은 종료되지 않는 테스트를 실패시킨다.

## 저장 → Terminal → Run Task

File → Save All 다음 Terminal → Run Task...을 누른다.



1. 01 Check tools를 실행하고 도구 버전을 확인한다.
2. 02 Simulate를 선택하고 종료까지 기다린다.
3. 03 Open waveform으로 생성된 VCD를 연다.

작업이 없으면 올바른 .code-workspace를 열었는지 확인한다.

# 실행 로그 확인

PROJECT → build → vscode → simulation.log를 연다.

LAB1\_PASS mux\_4x1 cases=64

정상 종료 시각: 640ns. PASS 문구와 검사 수를 함께 확인한다.

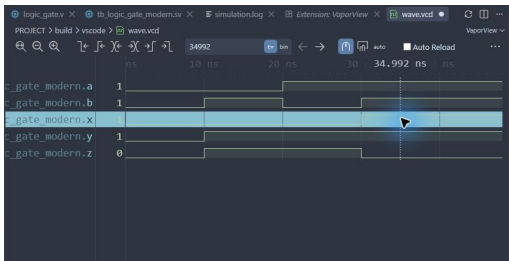
실패하면 compile.log → elaborate.log → simulation.log 순서로 원인을 찾는다.

코드 저장 → 02 Simulate 재실행 → 새 로그 확인 순서로 복귀한다.  
프로젝트 검증 안내

# VCD를 파형으로 열기

1. 03 Open waveform 또는 build/vscode/wave.vcd를 연다.
2. 텍스트로 열리면 탭 우클릭 → Reopen Editor With... → VaporView.
3. Netlist View에서 tb\_mux\_4x1을 펼친다.
4. 신호 i[3:0], s[1:0], z를 각각 두 번 눌러 추가한다.
5. Zoom to Fit를 누르고 Time Units를 ns로 맞춘다.
6. 전환 경계 대신 구간 중간에 커서를 두고 값을 읽는다.

# 파형 화면의 읽는 순서



위는 공통 파형 조작 예다. 이번 회로에서는 앞 장의 신호 이름을 선택한다.

i=1000인 320–360ns에서 s를 바꿨을 때 선택된 입력을 추적한다.

신호 이름 → 시간 구간 → 커서 값 → 예상 결과 순서로 비교한다.

## 실제 VCD의 구간 중간값

| 시간(ns) | i[3:0] | s[1:0] | z |
|--------|--------|--------|---|
| 5      | 0000   | 00     | 0 |
| 15     | 0000   | 01     | 0 |
| 35     | 0000   | 11     | 0 |
| 325    | 1000   | 00     | 1 |
| 635    | 1111   | 11     | 1 |

이 표는 실행된 VCD에서 읽은 값이다. 다중 비트는 이진수다.

표의 시간으로 파형 커서를 옮겨 직접 대조하고, 추가 경계 조건도 확인한다.

# 실험 전 레포트

1. 목적·포트·비트 폭·예상표와 동작 원리를 설명한다.
  2. 소스와 TB 링크, 코드 커밋, 도구 버전을 남긴다.
  3. 입력 조합·기대값·자동 비교·종료 조건을 설명한다.
  4. 자신의 PASS 로그와 파형 원본·캡처를 첨부한다.
  5. 시간 구간별 값을 해석하고 예상값과 비교한다.
  6. 오류·수정·재실행, 보드에서 확인할 입력과 출력을 적는다.
- 교재 캡처를 자신의 실행 결과로 제출하지 않는다.

# Vivado GUI에서 이어서 실습

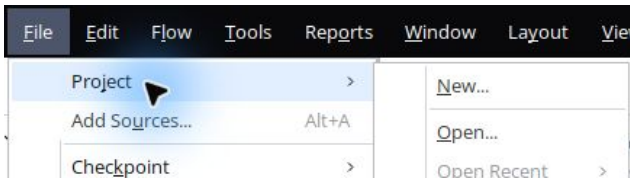
사전 시뮬레이션과 실험 전 레포트를 마친 뒤 Vivado 2026.1을 연다.

이후 단계는 메뉴와 버튼을 직접 누른다.

공통 클릭 화면은 첫 회로에서 실제 캡처했다. 이번 회로의 입력 파일과 top은 다음 슬라이드의 값을 따른다.

첫 회로의 상세 GUI 매뉴얼

## File → Project → New...



New Project 안내에서 Next. Project name: mux\_4x1

Project location: 자신의 clone 폴더 / vivado\_2026\_1/06\_mux4to1 / vivado

Create project subdirectory를 해제하고 Next. RTL Project와 Do not specify sources at this time을 선택한다.

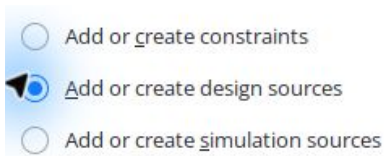
# 정확한 부품 선택

|                    |               |                |              |   |
|--------------------|---------------|----------------|--------------|---|
| Parts Boards       |               | Q              |              |   |
| Q: xc7s75fgga484-1 |               | (3 matches)    |              |   |
| Part               | I/O Pin Count | Available IOBs | LUT Elements | F |
| xc7s75fgga484-1    | 484           | 338            | 48000        | 9 |
| xc7s75fgga484-1IL  | 484           | 338            | 48000        | 9 |
| xc7s75fgga484-1Q   | 484           | 338            | 48000        | 9 |

Parts에서 xc7s75fgga484-1 검색 → 정확히 같은 행 선택 → Next.

요약에서 Spartan-7, fgga484, -1을 확인하고 Finish.

# RTL을 Design Sources에 추가



왼쪽 Add Sources → Add or create design sources → Next → Add Files.

등록할 RTL: mux\_4x1.v

Copy sources into project를 해제하고 Finish. VS Code와 같은 원본을 참조한다.

# Simulation Sources에 TB 추가

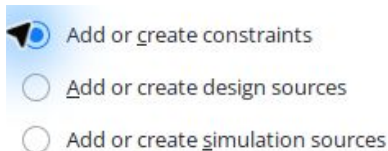
- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒  Add or create simulation sources

Add Sources → Add or create simulation sources → Next → Add Files.

tb\_mux\_4x1.sv를 선택한다. sim\_1을 유지한다.

Copy sources into project는 해제, Include all design sources for simulation은 체크 → Finish.

# XDC를 Constraints에 추가



Add Sources → Add or create constraints → Next → Add Files.

mux\_4x1.xdc 선택 → Copy constraints files into project 해제 → Finish.

소스 중복 등록이나 잘못된 종류로 추가한 파일이 없는지 확인한다.

# 두 top을 구분한다

Design Sources의 top: mux\_4x1

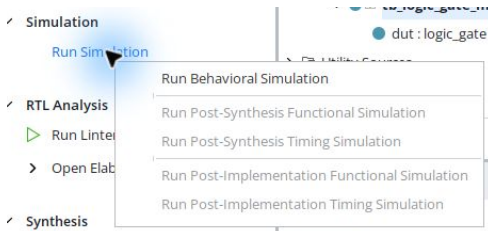
Simulation Sources → sim\_1의 top: tb\_mux\_4x1

Sources의 화살표를 눌러 계층을 펼친다. 굵은 이름이 top이다.

잘못 지정되었다면 올바른 모듈 우클릭 → Set as Top. 이미 top이면 해당 메뉴가 비활성인 것이 정상이다.

테스트벤치를 합성할 설계 top으로 지정하지 않는다.

# Run Behavioral Simulation



Run Simulation → Run Behavioral Simulation. 컴파일과 elaboration을 기다린다.

파형에서 신호를 선택하고 Zoom Fit, 시간 단위, 커서 값을 확인한다.

Tcl Console의 PASS 문구와 종료 시각을 확인한다. 오류면 Messages의 첫 오류로 돌아간다.

# VS Code 결과와 비교

두 실행은 같은 RTL과 자기검사 TB를 사용한다.

LAB1\_PASS mux\_4x1 cases=64

VS Code 로그: build/vscode/. GUI 로그:  
vivado/mux\_4x1.sim/sim\_1/behav/xsim/.

i=1000인 320-360ns에서 s를 바꿨을 때 선택된 입력을 추적한다.

로그·파형을 서로 다른 이름으로 보관하고 네 가지 정보인 입력·출력·  
시간·검사 수를 비교한다.

# 대상 부품·핀 제약 (1)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

| 포트   | PACKAGE_PIN |
|------|-------------|
| z    | L4          |
| i[3] | K4          |
| i[2] | N8          |
| i[1] | N4          |
| i[0] | N1          |
| s[1] | Y1          |

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

## 대상 부품·핀 제약 (2)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

| 포트   | PACKAGE_PIN |
|------|-------------|
| s[0] | W3          |

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

# Run Synthesis

- ✓ Synthesis
  - ▶ Run Synthesis
  - > Open Synthesized Design

File → Close Simulation → OK. Flow Navigator → Run Synthesis.

Launch Runs: 기본 디렉터리, Launch runs on local host, PC 자원에 맞는 jobs → OK.

Synthesis successfully completed를 확인한다. 실패했으면 다음 단계로 넘어가지 않는다.

# Run Implementation

 Synthesis successfully completed.

## Next

---

- ☒ Run Implementation
- ☐ Open Synthesized Design
- ☐ View Reports

합성 완료 창에서 Run Implementation → OK. Launch Runs에서 OK.  
완료 창을 닫았다면 Flow Navigator → Run Implementation.  
Implementation successfully completed가 나올 때까지 기다린다.

# Generate Bitstream

 Implementation successfully completed.

## Next

---

☐ Open Implemented Design

☒ Generate Bitstream

☐ View Reports

구현 완료 창에서 Generate Bitstream → OK → Launch Runs의 OK.

또는 Program and Debug → Generate Bitstream.

최종 상태 write\_bitstream Complete!를 확인한다. 파일 생성과 보드 기록은 다른 단계다.

# 생성 파일과 보고서

생성 bit: vivado/mux\_4x1.runs/impl\_1/mux\_4x1.bit

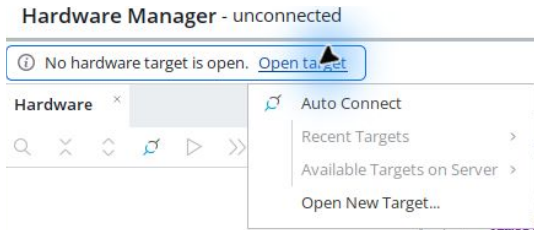
제작 실행 결과와 증빙은 프로젝트의 evidence/validation.json과 README  
검증표를 확인한다.

DRC 오류를 확인하고 CFGBVS / CONFIG\_VOLTAGE 경고는 실제 보드 구성  
전압과 대조한다.

클록 없는 조합회로의 setup/hold NA를 타이밍 검증 완료로 해석하지  
않는다.

사용한 커밋·bit 해시·DRC·타이밍 보고서를 결과와 함께 남긴다.

# 실제 보드 연결



Open Hardware Manager → Open target → Auto Connect.

KEY1-4=i[3:0], DIP1-2=s[1:0]. LED1=z.

장치가 없으면 보드 전원·USB JTAG·드라이버·VM USB 연결을 점검한다.

# Program Device와 동작 확인

1. 연결된 장치 이름이 xc7s75인지 확인한다.
2. 장치 우클릭 → Program Device.
3. 이번 프로젝트의 mux\_4x1.bit를 선택하고 Program.
4. 기록 완료를 확인한 뒤 입력을 바꾸고 출력과 예상값을 비교한다.
5. 기록 성공 화면과 실제 보드 동작은 각각 증빙한다.

제작 환경의 실제 보드 기록·촬영 검증 여부는 검증표를 따른다. 파일 생성만으로 보드 동작 성공을 선언하지 않는다.

# 사진과 시연 영상 촬영

KEY1-4=i[3:0], DIP1-2=s[1:0]. LED1=z.

사진에는 보드 연결과 입력·출력 위치가 함께 보이게 한다.

영상에는 입력을 조작하는 과정과 그에 따른 출력 변화를 담는다.

예상표의 정상·경계 조건을 직접 보여주고 회로 번호를 파일명에 적는다.

통합 영상이라면 각 회로의 타임스탬프를 레포트에서 연결한다.

# GitHub 결과 정리

1. reports/pre와 reports/post에 실험 전·후 레포트를 둔다.
  2. evidence/vscode와 evidence/vivado에 로그·파형·캡처를 구분한다.
  3. evidence/board/photos와 videos에 직접 촬영한 자료를 둔다.
  4. 큰 영상·bit는 배포 위치를 정하고 README와 레포트에서 연결한다.
  5. 웹에서 사진 표시와 영상 재생 또는 다운로드가 되는지 확인한다.
- 레포트 양식·예시·GitHub 안내

# 실험 후 레포트

1. Vivado 버전·part·top·핀 제약·코드 커밋을 기록한다.
2. VS Code와 Vivado의 입력·출력·시간·검사 수를 비교한다.
3. 합성·구현·bit 경로와 경고·수정 사항을 설명한다.
4. 장치 기록 화면·사진·영상과 해당 조건을 연결한다.
5. 예상값·두 시뮬레이션·실측의 일치 또는 차이 원인을 해석한다.
6. 미수행 항목은 미완료로 남기고 후속 확인을 적는다.

전체 목차 PDF 프로젝트로 돌아가기